

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHÓA ĐÀO TẠO QUỐC TẾ
BỘ MÔN KỸ THUẬT LẬP TRÌNH



CHUYÊN ĐỀ
HỆ THỐNG QUẢN LÝ
THU CHI CÁ NHÂN

Giảng viên hướng dẫn: CAO THỊ LUYỀN

Nhóm : 05

Lớp : CNTT VIỆT – ANH 2

Khoá : K66

Hà Nội, tháng 4 năm 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA ĐÀO TẠO QUỐC TẾ
BỘ MÔN KỸ THUẬT LẬP TRÌNH



CHUYÊN ĐỀ
HỆ THỐNG QUẢN LÝ
THU CHI CÁ NHÂN

Giảng viên hướng dẫn : CAO THỊ LUYỀN

Sinh viên thực hiện: Hoàng Nhất Nam
Phạm Tuấn Kiệt
Nguyễn Văn Vinh
Bùi Tiến Đạt
Đào Ngọc Minh

Lớp : CNTT Việt – Anh 2
Khoá : K66

Hà Nội, tháng 4 năm 2026

LỜI MỞ ĐẦU

Trong lĩnh vực lập trình hiện đại, việc viết một chương trình có thể chạy đúng mới chỉ là bước khởi đầu. Thách thức thực sự nằm ở việc làm thế nào để mã nguồn đó dễ hiểu, dễ bảo trì và có khả năng mở rộng. Đứng trước yêu cầu đó, khái niệm về "Hàm" và "Tổ chức chương trình" đóng vai trò là nền tảng cốt lõi trong tư duy thiết kế phần mềm. Việc nắm vững cách phân chia mô-đun (Modularization) không chỉ giúp lập trình viên quản lý logic phức tạp một cách hiệu quả mà còn tối ưu hóa khả năng làm việc nhóm. Nhận thức được tầm quan trọng này, nhóm chúng em đã lựa chọn đề tài "**Hệ thống quản lý và thu chi cá nhân**" để thực hiện bài báo cáo. Thông qua việc xây dựng chương trình này, nhóm mong muốn áp dụng các nguyên tắc thiết kế hàm tối ưu, phân tách thành phần hệ thống và rèn luyện kỹ năng tổ chức mã nguồn khoa học.

LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành đến cô “Cao Thị Luyện” bộ môn Kỹ thuật lập trình đã hướng dẫn và tạo điều kiện cho chúng em thực hiện đề tài này. Nhờ những kiến thức quý báu về cách tổ chức chương trình và tư duy mô-đun hóa mà thầy/cô truyền đạt, nhóm đã có cơ hội áp dụng lý thuyết vào thực tiễn để xây dựng nên chương trình “Hệ thống quản lý và thu chi cá nhân”. Chúng em nhận thấy đây không chỉ là một bài tập học thuật đơn thuần, mà là một cơ hội quý báu để chúng em được thử thách bản thân. Thông qua đề tài này, nhóm đã có điều kiện để trau dồi kiến thức chuyên môn, hiểu sâu hơn về cách thiết kế hàm tối ưu, tư duy mô-đun hóa và cách tổ chức mã nguồn khoa học. Đồng thời, chúng em cũng được phát triển kỹ năng mềm qua việc rèn luyện khả năng làm việc nhóm, kỹ năng thuyết trình và tư duy phân tích, giải quyết vấn đề thực tế. Chính nhờ sự gợi mở và tạo điều kiện từ phía thầy/cô, chúng em đã có một môi trường học tập năng động, giúp biến những lý thuyết khô khan trở thành những sản phẩm thực tế có giá trị. Những kỹ năng và kinh nghiệm tích lũy được từ quá trình thực hiện đề tài này chắc chắn sẽ là hành trang quan trọng cho chúng em trong những chặng đường phát triển sự nghiệp sau này. Chúng em xin chân thành cảm ơn cô!

Bảng phân công nhiệm vụ

STT	Họ và tên sinh viên	Mã sinh viên	Nhiệm vụ được phân công
1	Hoàng Nhật Nam	252601071	Nhóm trưởng. Viết và kiểm thử chương trình
2	Nguyễn Văn vinh	252631097	Tạo và thiết kế powerpoint
3	Phạm Tuấn Kiệt	252601060	Tìm hiểu nội dung và làm báo cáo
4	Bùi Tiên Đạt	252601031	Tìm hiểu về nội dung và lý thuyết Của hàm và chương trình
5	Đào Ngọc Minh	252601067	Tìm hiểu về nội dung và cách tổ chức chương trình

MỤC LỤC

PHẦN I : PHẦN MỞ ĐẦU	1
1.1. Lý do chọn đề tài và mục tiêu nghiên cứu	1
1.2. Mục tiêu của báo cáo.....	1
1.3. Đối tượng và phạm vi nghiên cứu.....	1
1.4. Phương pháp thực hiện	2
1.5. Giới thiệu chương trình xây dựng.....	2
PHẦN II : CƠ SỞ LÝ THUYẾT	4
2.1. Khái niệm và vai trò của hàm	4
2.2. Thiết kế hàm : hàm tốt và hàm xấu	4
2.3. Tham số và giá trị trả về.....	5
PHẦN III : LIÊN HỆ GIỮA LÝ THUYẾT VÀ CODE.....	7
3.1 Minh họa thiết kế hàm trong chương trình	7
3.2. So sánh giữa trước và sau khi cải tiến chương trình.....	8
PHẦN IV : CẤU TRÚC TỔ CHỨC CHƯƠNG TRÌNH.....	10
4.1. Cấu trúc tổ chức file.....	10
4.2. Danh sách các hàm chức năng trong chương trình	14
PHẦN V : PHÂN TÍCH VÀ ĐÁNH GIÁ	15
5.1. Điểm mạnh và Điểm yếu của chương trình	15
5.2. Hướng cải tiến và Phát triển.....	15
PHỤ LỤC	16
DANH MỤC TÀI LIỆU THAM KHẢO	17

PHẦN I : PHẦN MỞ ĐẦU

1.1. Lý do chọn đề tài và mục tiêu nghiên cứu

Trong chương trình học phần kỹ thuật lập trình , nội dung về “Hàm và Tổ chức chương trình” là kiến thức cốt lõi giúp sinh viên hình thành tư duy lập trình chuyên nghiệp. Thay vì viết mã nguồn rời rạc, việc nắm vững cách xây dựng hệ thống theo module là kỹ năng bắt buộc để phát triển các dự án thực tế.

Được sự phân công của giảng viên hướng dẫn, nhóm chúng em đã lựa chọn và thực hiện đề tài “Hệ thống quản lý thu chi cá nhân ” nhằm các mục đích cụ thể sau:

- Vận dụng kiến thức: Hiện thực hóa các lý thuyết về thiết kế hàm, quản lý biến và chia tách module (Modularization) đã được học trên lớp.
- Rèn luyện kỹ năng thực tế: Đối mặt và giải quyết các bài toán về logic dữ liệu, gỡ lỗi và tối ưu hóa mã nguồn trong một chương trình hoàn chỉnh.
- Nâng cao khả năng làm việc nhóm: Thực hành quy trình phối hợp, phân chia công việc và ghép nối các thành phần mã nguồn giữa các thành viên.

Thông qua đề tài này, nhóm mong muốn củng cố nền tảng kỹ thuật lập trình và hoàn thành tốt nhiệm vụ học tập được thầy/cô giao phó.

1.2. Mục tiêu của báo cáo

Báo cáo này được xây dựng nhằm đạt được các mục tiêu trọng tâm sau:

- Hệ thống hóa lý thuyết: Tổng hợp và làm rõ các khái niệm cốt lõi về hàm, cơ chế truyền tham số, giá trị trả về, cũng như phân biệt giữa các nguyên tắc thiết kế hàm "tốt" và "xấu".
- Vận dụng thực hành: Xây dựng một chương trình sử dụng cấu trúc hàm và tổ chức được chương trình một cách hoàn chỉnh, đảm bảo tính ổn định và logic trong xử lý.
- Phát triển kỹ năng: Rèn luyện khả năng thiết kế hệ thống có cấu trúc, nâng cao kỹ năng làm việc nhóm qua việc phân chia module và trình bày báo cáo chuyên nghiệp theo yêu cầu của học phần.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Tập trung vào cú pháp và logic của ngôn ngữ lập trình C; áp dụng kỹ thuật Modularization để phân tách chương trình thành các file tiêu đề (.h) và file thực thi (.c); quản lý phạm vi hoạt động của biến (cục bộ và toàn cục) để tối ưu hóa bộ nhớ.

Phạm vi nghiên cứu: Dự án tập trung vào việc xử lý logic dữ liệu thông qua cấu trúc mảng và bản ghi (struct), xử lý thuật toán sắp xếp và tìm kiếm sau đó lưu trữ kết quả đầu ra vào tệp tin văn bản **output.txt**.

1.4. Phương pháp thực hiện

Để hoàn thành dự án, nhóm đã áp dụng quy trình phát triển phần mềm rút gọn gồm 4 bước:

1. Phân tích yêu cầu: Khảo sát bài toán thực tế để xác định danh sách các chức năng cần thiết (input, process, output).
2. Thiết kế hệ thống: Phác thảo sơ đồ phân rã chức năng. Tại bước này, nhóm tập trung xác định các hàm cần thiết, tên hàm, kiểu dữ liệu trả về và các tham số truyền vào.
3. Lập trình và đóng gói: Tiến hành viết mã nguồn theo hướng tách biệt giữa giao diện người dùng (Menu) và các hàm xử lý logic để đảm bảo tính độc lập.
4. Kiểm thử và tối ưu: Chạy thử chương trình với các bộ dữ liệu khác nhau, sau đó tiến hành tái cấu trúc (refactoring) những đoạn code chưa tối ưu dựa trên tiêu chuẩn "hàm tốt vs hàm xấu".

1.5. Giới thiệu chương trình xây dựng

Tên chương trình: Hệ thống quản lý thu chi cá nhân

Số lượng hàm sử dụng: 10 hàm xử lý chính (được module hóa qua các file .c và .h)

Các tính năng cốt lõi:

1. Chức năng Nhập liệu (**Hàm nhapgiaodich & docFile**):
 - Hỗ trợ hai phương thức: Nhập trực tiếp từ bàn phím hoặc đọc dữ liệu tự động từ tệp tin **input.txt**.
 - Sử dụng cấu trúc dữ liệu **giaodich** để lưu trữ chi tiết: tên khoản, số tiền, loại (thu/chi) và thời gian.
2. Chức năng Hiển thị (**Hàm indanhsach**):
 - Xuất toàn bộ danh sách giao dịch ra màn hình console theo dạng bảng định dạng sẵn, giúp người dùng dễ dàng theo dõi.
3. Chức năng Tính toán tài chính (**Hàm tinh tongThu, tinh tongChi, tinh soDu**):
 - Tự động lọc và tính tổng các khoản "thu", tổng các khoản "chi".
 - Tính toán số dư thực tế dựa trên chênh lệch thu-chi.

4. Chức năng Thống kê theo thời gian (Hàm **thongketheothang**):

- Cho phép người dùng nhập tháng và năm để xem lại các giao dịch phát sinh trong khoảng thời gian đó kèm theo tổng tiền cụ thể.

5. Chức năng Tìm kiếm (Hàm **timkiemtheoten**):

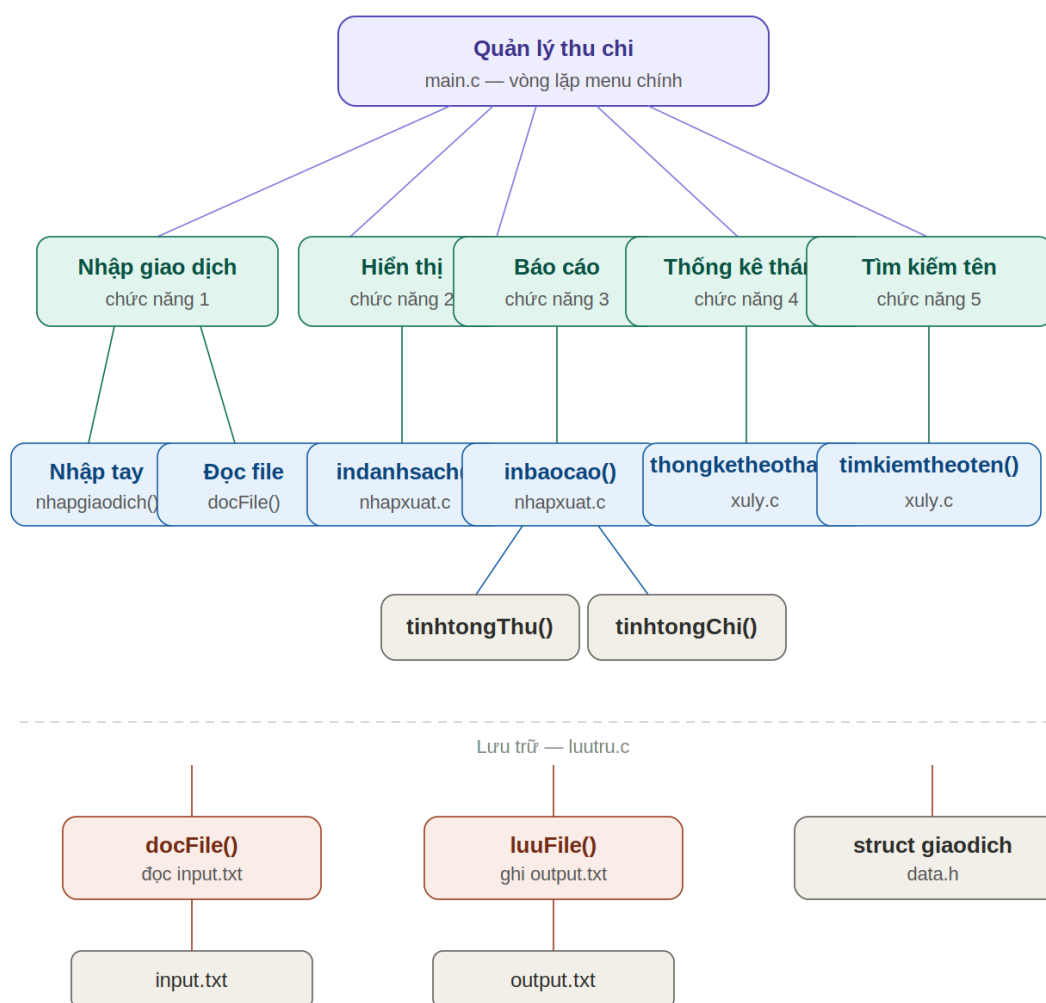
- Truy xuất nhanh các giao dịch dựa trên từ khóa tên khoản do người dùng nhập vào.

6. Chức năng Lưu trữ bền vững (Hàm **luuFile**):

- Khi thoát chương trình, toàn bộ dữ liệu hiện hành sẽ được tự động kết xuất và lưu lại vào tệp tin **output.txt** để phục vụ việc tra cứu lâu dài.

7. Giao diện điều khiển (Hàm **hienthimenu**):

- Hệ thống menu lựa chọn từ 0 đến 7 giúp người dùng tương tác thuận tiện với tất cả các chức năng của chương trình.



PHẦN II : CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và Vai trò của Hàm

- Định nghĩa: Hàm là một khối lệnh thực hiện một công việc cụ thể, được đặt tên và có thể tái sử dụng nhiều lần.
- Vai trò: Giúp chia nhỏ chương trình (Divide and Conquer), dễ quản lý, dễ gỡ lỗi và tránh lặp code.
- Khi nào nên/không nên dùng: Dùng khi một thao tác lặp lại nhiều lần hoặc logic quá phức tạp. Không nên dùng cho các thao tác quá đơn giản (như cộng 2 số chỉ dùng 1 lần) để tránh làm chậm chương trình do chi phí gọi hàm.
- Ví dụ minh họa về hàm :

```
int  tinhTongThu (int n ,giaodich *t) {
    int i;
    int tongThu = 0;
    for (i=0;i<n;i++) {
        if (strcmp(t[i].loai, "thu") == 0) {
            tongThu += t[i].sotien;
        }
    }
    return tongThu;
}
```

2.2. Thiết kế Hàm: Hàm tốt vs Hàm xấu

- Hàm tốt: Có tên gọi nhớ, chỉ thực hiện duy nhất một nhiệm vụ (Single Responsibility), các tham số đầu vào rõ ràng.
- Hàm xấu: Quá dài (hàng trăm dòng), thực hiện nhiều việc cùng lúc, lạm dụng biến toàn cục khiến hàm khó kiểm soát.

- ví dụ :
Hàm tốt :

```
int tinhSoDu (int thu, int chi) {
    return thu-chi;
}
```

Hàm xấu :

```
void xử_lý(int x) {
    if (x > 0) {
        printf("Thu: %d", x);
        tong_thu += x;
    } else {
        printf("Chi: %d", -x);
        tong_chi += -x;
    }
    FILE *f = fopen("data.txt", "a");
    fprintf(f, "%d\n", x);
    fclose(f);
}
```

Tại sao hàm này lại "Xấu"?

1. Tên hàm vô nghĩa: Tên xử_lý không cho biết hàm này thực sự làm gì. Trong lập trình chuyên nghiệp, nên dùng tiếng Anh không dấu hoặc tiếng Việt rõ ràng như nhập_giao_dich.
2. Vi phạm nguyên tắc đơn nhiệm (Single Responsibility): Một hàm tốt chỉ nên làm đúng một việc. Hàm này đang làm quá nhiều thứ: kiểm tra dữ liệu, in ra màn hình, tính toán tổng, và ghi cả vào file.
3. Khó bảo trì: Nếu sau này bạn muốn thay đổi cách ghi file, bạn phải sửa lại toàn bộ hàm này. Nếu muốn tính tổng mà không in ra màn hình, bạn cũng không thể tái sử dụng hàm này được.
4. Lạm dụng biến toàn cục: Việc thay đổi trực tiếp tong_thu và tong_chi bên trong hàm khiến code dễ bị lỗi logic nếu chương trình lớn dần lên.

2.3. Tham số và Giá trị trả về

- Giải thích về cách truyền giá trị vào hàm và cách dùng lệnh return để nhận kết quả.
- Phân biệt truyền tham trị (Value) và truyền tham chiếu (Pointer) để thay đổi giá trị biến gốc.

- Ví dụ :

```
int tinhTongChi (int n, giaodich *t) {  
    int i;  
    int tongChi = 0;  
    for (i=0;i<n;i++) {  
        if (strcmp (t[i].loai,"chi") == 0) {  
            tongChi += t[i].sotien;  
        }  
    }  
    return tongChi;  
}
```

PHẦN III : LIÊN HỆ GIỮA LÝ THUYẾT VÀ CODE

3.1. Minh họa thiết kế hàm trong chương trình

- Trong dự án này, nhóm đã vận dụng các kiến thức lý thuyết về hàm để xây dựng hệ thống quản lý thu chi. Mỗi hàm được thiết kế để thực hiện một nhiệm vụ duy nhất, tuân thủ nguyên tắc Single Responsibility.

- Ví dụ về hàm xử lý logic (**xuly.c**): Hàm tinhTongThu nhận vào danh sách giao dịch và trả về một giá trị nguyên đại diện cho tổng số tiền.

```
int tinhTongThu (int n ,giaodich
*t) {
    int i;
    int tongThu = 0;
    for (i=0;i<n;i++) {
        if (strcmp(t[i].loai,
"thu") == 0) {
            tongThu +=
t[i].sotien;
        }
    }
}
```

- Ví dụ về hàm tương tác tệp (**luutru.c**): Hàm lưuFile sử dụng con trỏ **FILE** * để ghi dữ liệu đã được định dạng vào tệp tin output.txt.

```
void lưuFile (FILE *a,int n, giaodich *t) {
    int i;
    fprintf(a,"\n%-5s %-25s %-10s %-10s %-12s\n", "STT", "Ten
khoan", "Loai", "So tien", "Ngay");
    for (int i = 0; i < n; i++) {
        fprintf(a,"%-5d %-25s %-10s %-10d %02d/%02d/%04d\n",
i + 1,
t[i].tenkhoan,
t[i].loai,
t[i].sotien,
t[i].ngay,
t[i].thang,
t[i].nam );
    }
}
```

3.2. So sánh giữa trước và sau khi cải tiến chương trình

- Để làm rõ lợi ích của việc tổ chức chương trình theo hàm và module, nhóm thực hiện phân tích đối chiếu giữa phương pháp viết mã tập trung (Monolithic) và phương pháp phân tách chức năng (Modular) đang áp dụng trong dự án.

a. Cách tiếp cận cũ (Mã nguồn tập trung): Trước khi cải tiến, toàn bộ logic từ nhập liệu, xử lý thuật toán đến hiển thị đều được viết dồn vào một hàm main duy nhất.

- Đặc điểm: Mã nguồn kéo dài liên tục, các biến cục bộ lẫn lộn gây khó khăn cho việc kiểm soát luồng dữ liệu.
- Hệ quả: Khi cần thay đổi một logic nhỏ (ví dụ: thay đổi cách tính số dư), người lập trình phải rà soát lại toàn bộ chương trình, dễ dẫn đến lỗi hệ thống và rất khó để tái sử dụng mã nguồn cho các mục đích khác.

```
int main() {
    int n, tongThu = 0, tongChi = 0;
    giaodich t[100];
    printf("So giao dich: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Ten khoan: "); scanf("%s", t[i].tenkhoan);
        printf("So tien: ");    scanf("%d", &t[i].sotien);
        printf("Loai: ");      scanf("%s", t[i].loai);
    }
    for (int i = 0; i < n; i++)
        printf("%-5d %-20s %-5s %d\n", i+1, t[i].tenkhoan,
t[i].loai, t[i].sotien);
    for (int i = 0; i < n; i++) {
        if (strcmp(t[i].loai, "thu") == 0) tongThu +=
t[i].sotien;
        else tongChi += t[i].sotien;
    }
    printf("Tong thu: %d\nTong chi: %d\nSo du: %d\n",
tongThu, tongChi, tongThu - tongChi);
    FILE *f = fopen("output.txt", "w");
    for (int i = 0; i < n; i++)
        fprintf(f, "%s %d %s\n", t[i].tenkhoan, t[i].sotien,
t[i].loai);
    fclose(f);
    .....
    return 0;
}
```

b. Cách tiếp cận cải tiến (Mã nguồn theo module): Dựa trên yêu cầu của đề tài, nhóm đã tiến hành "chia để trị" bằng cách tách các chức năng thành những hàm độc lập và đóng gói vào các tệp tin chuyên biệt như **nhapxuat.c**, **xuly.c**, và **luutru.c**.

- Ví dụ minh họa sự gọn gàng trong hàm main: Thay vì chứa hàng trăm dòng code xử lý, hàm main hiện nay chỉ đóng vai trò điều hướng thông qua cấu trúc switch-case:

```
        case 5:
            if (a == NULL) {
                printf("Chua co khoan giao dich duoc ghi\n");
            } else {
                printf("So du: %d\n", tinhsoDu(tinh tongThu(n,a), tinh tongChi(n,a))); }
            break;

        case 6:
            if (a == NULL) {
                printf("Chua co khoan giao dich duoc ghi\n");
            } else {
                int thang,nam;
                printf("Nhap thang/nam: ");
                scanf("%d%d",&thang,&nam);
                thongketheothang(n,a,thang,nam); }
            break;
```

c. Ưu điểm của cấu trúc hiện tại:

- Tính tường minh: Mỗi hàm chỉ thực hiện một nhiệm vụ (ví dụ: **tinh tongThu** chỉ để tính tiền thu, **luuFile** chỉ để ghi dữ liệu) giúp mã nguồn cực kỳ dễ đọc.
- Dễ dàng bảo trì: Khi có lỗi phát sinh trong việc đọc file, nhóm chỉ cần tập trung kiểm tra tệp **luutru.c** mà không lo ảnh hưởng đến logic tính toán ở **xuly.c**.
- Tối ưu hóa tài nguyên: Việc truyền tham số qua con trỏ **giaodich *t** giúp các hàm truy cập trực tiếp vào dữ liệu mà không cần tạo bản sao, giúp chương trình chạy nhanh và tiết kiệm bộ nhớ.

PHẦN IV : CẤU TRÚC TỔ CHỨC CHƯƠNG TRÌNH

4.1. Cấu trúc tổ chức file

- Chương trình được xây dựng theo phương pháp lập trình mô-đun, chia tách rõ ràng giữa định nghĩa dữ liệu, giao diện và logic xử lý:

- **data.h**: Định nghĩa cấu trúc dữ liệu giaodich dùng chung cho toàn bộ hệ thống.

```
#ifndef DATA_H
#define DATA_H
    typedef struct
    {
        char tenkhoan[100];
        int sotien;
        char loai[10];
        int ngay;
        int thang;
        int nam;
    } giaodich;
#endif
```

- **nhapxuat.c/.h**: Quản lý giao diện menu và các hàm nhập/xuất dữ liệu trực tiếp với người dùng.

```
#ifndef NHAPXUAT_H
#define NHAPXUAT_H
#include "data.h"
int hienthimenu();
void nhapgiaodich(giaodich *t, int n);
void indanhsach(int n, giaodich *t);
void inbaocao(int n, giaodich *t);
```



```

void nhapgiaodich (giaodich *t, int n) {
    int i;
    getchar();
    for (i=0;i<n;i++) {
        printf("STT: %d\n",i+1);
        printf("Nhap ten khoan: ");
        fgets(t[i].tenkhoan,sizeof(t[i].tenkhoan),stdin);
        t[i].tenkhoan[strlen(t[i].tenkhoan) -1 ] = 0;
        printf("Nhap so tien: "); scanf("%d",&t[i].sotien);
        getchar();
        printf("Loai(thu/chi): ");
        fgets(t[i].loai,sizeof(t[i].loai),stdin);
        t[i].loai[strlen(t[i].loai) -1 ] = 0;
        printf("Nhap ngay/thang/nam: ");
        scanf("%d%d%d",&t[i].ngay,&t[i].thang,&t[i].nam);
        getchar();
        printf("\n");
    }
}

```

- **xuly.c /.h:** Chứa các thuật toán cốt lõi như tính tổng thu, tổng chi, số dư và các hàm tìm kiếm, thống kê.

```

#ifndef XULY_H #define XULY_H #include "data.h"
int tinh tongThu (int n ,giaodich *t);
int tinh tongChi (int n, giaodich *t);
int tinh soDu (int thu, int chi);
void thongke theo thang (int n, giaodich *t, int thang,
int nam);
void timkiem theoten (int n, giaodich *t, char
key[]);#endif

```

```

void thongketheothang(int n, giaodich *t, int thang, int nam) {
    int i, j = 0, check = 0;
    for (i = 0; i < n; i++) {
        if (t[i].nam == nam && t[i].thang == thang) {
            j++;
            if (check == 0) {
                printf("\n===== THONG KE THANG %02d/%04d
=====\\n", thang, nam);
                printf("%-5s %-20s %-10s %-12s %-10s\\n", "STT",
"Ten khoan", "Loai", "So tien", "Ngay");
                check = 1;
            }
            printf("%-5d %-20s %-10s %-12d %02d/%02d/%04d\\n",
                j, t[i].tenkhoan, t[i].loai, t[i].sotien,
                t[i].ngay, t[i].thang, t[i].nam);
        }
    }
    if (check == 0) {
        printf("\\nKhong co giao dich nao trong thang
%02d/%04d\\n", thang, nam);
    }

    printf("=====\\n");
}

```

- **luutru.c /.h**: Đảm nhiệm chức năng tương tác với tệp tin (**input.txt** và **output.txt**) để lưu trữ dữ liệu bền vững.

```

void luuFile (FILE *a,int n, giaodich *t) {
    int i;
    fprintf(a,"\n%-5s %-25s %-10s %-10s %-12s\n", "STT",
"Ten khoan", "Loai", "So tien", "Ngay");
    fprintf(a,"-----\n");
    for (int i = 0; i < n; i++) {
        fprintf(a,"%-5d %-25s %-10s%10d%02d/%02d/%04d\n",
i + 1,
t[i].tenkhoan,
t[i].loai,
t[i].sotien,
t[i].ngay,
t[i].thang,
t[i].nam );
    }
}

```

```

#ifndef LUUTRU_H
#define LUUTRU_H
#include "data.h"
#include <stdio.h>
void luuFile (FILE *a,int n, giaodich *t);
void docFile (FILE *a ,int n, giaodich *t);
#endif

```

- **main.c:** Điểm khởi đầu của chương trình, đóng vai trò điều hướng các chức năng thông qua menu.

```

#include "data.h"
#include "luutru.h"
#include "nhapxuat.h"
#include "xuly.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int n, chon;
    giaodich *a = NULL;
    do {
        chon = hienthimenu();
        switch (chon) {
            case 1: /* Nhap lieu: truc tiep hoac doc tu
input.txt */ break;
            case 2: /* Hien thi danh sach giao dich */
break;
            case 3: /* In bao cao tong thu, tong chi */
break;
            case 4: /* Thong ke theo thang/nam */
break;
            case 5: /* Tim kiem giao dich theo ten khoan
*/
break;
            case 0: /* Luu output.txt va thoat */
break;
            default: printf("Lua chon khong hop le!\n");
        }
    } while (chon != 0);
    free(a);
    return 0;
}

```

4.2. Danh sách các hàm chức năng trong chương trình

STT	Tên hàm	Chức năng	Tham số	Giá trị trả về
1	hienthimenu	Hiển thị các lựa chọn chức năng	Không	int (lựa chọn)
2	tinhtongThu	Tính tổng số tiền loại "thu", "chi"	int n, giaodich *t	int (tổng thu)
3	tinhsuDư	Tính chênh lệch thu - chi	Int thu, int chi	int (số dư)
4	thongketheothang	Lọc giao dịch theo tháng/năm	int n, giaodich *t, int m, int y	Không
5	docFile	Đọc/lưu dữ liệu từ tệp input.txt	FILE *a, int n, giaodich *t	Không
6	tinhtongChi	Tính tổng số tiền thuộc loại "chi"	int n, giaodich *t	int (tổng chi)
7	nhapgiaodich	Nhập thông tin giao dịch từ bàn phím	giaodich *t, int n	Không
8	indanhsach	Hiển thị toàn bộ danh sách giao dịch ra màn hình	int n, giaodich *t	Không
9	timkiemtheoten	Tìm kiếm giao dịch theo từ khóa tên khoản	int n, giaodich *t, char key[]	Không
10	luuFile	Ghi toàn bộ dữ liệu giao dịch vào tệp output.txt	FILE *a, int n, giaodich *t	Không

PHẦN V : PHÂN TÍCH VÀ ĐÁNH GIÁ

5.1. Điểm mạnh và Điểm yếu của chương trình

- Điểm mạnh :

- Tổ chức mã nguồn khoa học: Việc chia nhỏ thành các tệp .c và .h giúp mã nguồn dễ đọc, dễ quản lý và thuận tiện cho việc gỡ lỗi (debug).
- Xử lý tệp tin hiệu quả: Chương trình có khả năng tự động đọc dữ liệu từ input.txt và kết xuất báo cáo định dạng bảng ra output.txt khi thoát.
- Thái độ: Rèn luyện tính tỉ mỉ trong việc định dạng dữ liệu xuất ra tệp tin để báo cáo có tính thẩm mỹ và dễ tra cứu.

- Điểm yếu :

- Dữ liệu vẫn được quản lý trên mảng động, nếu số lượng bản ghi lên tới hàng triệu thì tốc độ tìm kiếm tuyến tính có thể bị ảnh hưởng.
- Giao diện tương tác qua Console còn đơn giản, chưa có màu sắc nổi bật để phân biệt nhanh giữa "thu" và "chi".

5.2. Hướng cải tiến và Phát triển

- Tối ưu cấu trúc dữ liệu: Nghiên cứu áp dụng Danh sách liên kết (Linked List) để việc thêm/xóa giao dịch trở nên linh hoạt hơn mà không cần cập phát lại toàn bộ mảng bộ nhớ.

- Bảo mật thông tin: Tích hợp thêm chức năng đăng nhập và mã hóa dữ liệu trong tệp tin để bảo vệ thông tin tài chính cá nhân

- Giao diện: Xây dựng giao diện đồ họa (GUI) hoặc sử dụng các thư viện như ncurses để cải thiện trải nghiệm người dùng.

5.3. Bài học kinh nghiệm từ dự án

- Quá trình thực hiện đề tài theo sự phân công của giảng viên đã giúp nhóm rút ra những bài học quan trọng:

- Kỹ thuật: Hiểu sâu hơn về cách truyền tham số bằng con trỏ để tối ưu hóa hiệu suất chương trình.

- Tổ chức: Nhận thấy tầm quan trọng của việc quy hoạch các tệp Header (.h) ngay từ đầu để tránh xung đột mã nguồn khi làm việc nhóm.

- Thái độ: Rèn luyện tính tỉ mỉ trong việc định dạng dữ liệu xuất ra tệp tin để báo cáo có tính thẩm mỹ và dễ tra cứu.

PHỤ LỤC

PHỤ LỤC A: CẤU TRÚC DỮ LIỆU VÀ GIAO DIỆN

A.1. Định nghĩa cấu trúc dữ liệu (data.h) : Đây là thành phần cốt lõi lưu trữ thông tin của mỗi giao dịch tài chính.

```
typedef struct { char tenkhoan[100]; int sotien; char loai[10];  
int ngay; int thang; int nam;} giaodich;
```

A.2. Hệ thống menu điều khiển (nhapxuat.c) : Danh sách các tính năng được cung cấp cho người dùng trong chương trình.

1. Nhập giao dịch (Trực tiếp hoặc từ file input.txt).
2. Hiện thị danh sách giao dịch hiện có.
3. Tính toán tổng thu nhập.
4. Tính toán tổng chi tiêu.
5. Kiểm tra số dư hiện tại.
6. Thống kê giao dịch theo tháng/năm cụ thể.
7. Tìm kiếm giao dịch theo tên khoản

PHỤ LỤC B: DỮ LIỆU MINH HỌA VÀ KẾT QUẢ

B.1. Dữ liệu đầu vào mẫu (input.txt) : Dữ liệu được chuẩn bị sẵn để kiểm tra khả năng đọc tệp của hệ thống.

```
3  
An sang  
20000  
chi  
10 4 2026  
Luong thang  
5000000  
thu  
1 4 2026  
Cafe  
30000  
chi  
9 4 2026
```

B.2. Kết quả xuất tệp báo cáo (output.txt) : Báo cáo tài chính được chương trình tự động kết xuất sau khi người dùng kết thúc phiên làm việc.

STT	Ten khoan	Loai	So tien	Ngay
1	An sang	chi	20000	10/04/2026
2	Luong thang	thu	5000000	01/04/2026
3	Cafe	chi	30000	09/04/2026

DANH MỤC TÀI LIỆU THAM KHẢO

Tiếng Việt:

1. Giáo trình Kỹ thuật lập trình C, Khoa Công nghệ thông tin, Nhà xuất bản Giáo dục Việt Nam, 2022.
2. Lập trình nâng cao với ngôn ngữ C, Nguyễn Hữu Tuấn, Nhà xuất bản Khoa học và Kỹ thuật, 2021.
3. Tài liệu hướng dẫn thực hành hàm và cấu trúc dữ liệu, Bộ môn Kỹ thuật máy tính, 2023.

Tiếng Anh:

1. Kernighan, B. W., & Ritchie, D. M., The C Programming Language, Second Edition, Prentice Hall, 1988.
2. Prata, S., C Primer Plus, Sixth Edition, Addison-Wesley Professional, 2013.
3. Standard C Library Documentation, ISO/IEC 9899:2011 (C11 standard), **.